Les Pointeurs en Langage C

Par Robert DIASSÉ

Introduction

Un pointeur en C est une **variable spéciale** qui stocke l'adresse mémoire d'une autre variable. Grâce aux pointeurs, vous pouvez accéder directement aux données en mémoire, ce qui rend leur utilisation très puissante, mais également délicate.

---

1. Pourquoi utiliser des pointeurs ?

- **Efficacité** : Permettent de manipuler directement les données en mémoire sans les copier.
- **Passage par référence** : Facilite la modification de valeurs dans des fonctions.
- **Gestion dynamique de mémoire** : Indispensable pour allouer de la mémoire pendant l'exécution avec `malloc` ou `calloc`.
- **Structures complexes** : Utiles pour manipuler des tableaux, chaînes de caractères, et structures.

---

2. Déclaration et Initialisation

a) Déclarer un pointeur

Un pointeur est déclaré avec l'opérateur `*` :

```
int *ptr; // Pointeur vers un entier
```

b) Initialiser un pointeur

Un pointeur doit être initialisé avec l'adresse d'une variable. L'opérateur `&` permet de récupérer l'adresse d'une variable :

```
int x = 10;
int *ptr = &x; // ptr pointe vers l'adresse de x
```

c) Accéder à la valeur pointée

L'opérateur `*` (déréférencement) permet d'accéder à la valeur stockée à l'adresse pointée :

```
printf("Valeur de x : %d\n", *ptr); // Affiche 10
```

---

3. Visualisation : Les pointeurs et la mémoire

Prenons l'exemple suivant :

```
int x = 42;
int *ptr = &x;
```

Variable	Adresse Mémoire	Valeur Stockée
`x`	0x7ffeab12	42
`ptr`	0x7ffeab16	0x7ffeab12 (adresse de `x`)

- `x` contient la valeur 42.
- `ptr` contient l'adresse de `x` (par exemple `0x7ffeab12`).
- `*ptr` donne la valeur stockée à l'adresse pointée (42).

---

4. Pointeurs et Tableaux

Un tableau est en réalité un pointeur vers son premier élément.

a) Accéder aux éléments d'un tableau avec des pointeurs

```
int tab[3] = {10, 20, 30};
int *ptr = tab; // Le pointeur pointe sur le premier élément

printf("%d\n", *ptr);     // Affiche 10
printf("%d\n", *(ptr+1)); // Affiche 20
printf("%d\n", *(ptr+2)); // Affiche 30
```

b) Passer un tableau à une fonction avec un pointeur

```
void afficherTableau(int *t, int taille) {
    for (int i = 0; i < taille; i++) {
        printf("%d ", *(t + i));
    }
}

int main() {
```

```
    int tab[3] = {10, 20, 30};
    afficherTableau(tab, 3); // Passe l'adresse du tableau
    return 0;
}
```

---

5. Pointeurs et Fonctions

a) Passage par valeur

Avec le passage par valeur, une copie de la variable est envoyée à la fonction. Les modifications dans la fonction n'affectent pas l'original :

```
void incrementer(int x) {
    x = x + 1;
}

int main() {
    int a = 10;
    incrementer(a);
    printf("%d\n", a); // Affiche 10 (pas modifié)
    return 0;
}
```

b) Passage par référence avec un pointeur

Avec un pointeur, on modifie directement la variable originale :

```
void incrementer(int *x) {
    *x = *x + 1; // Modifie la valeur pointée
}

int main() {
    int a = 10;
    incrementer(&a); // Passe l'adresse de a
    printf("%d\n", a); // Affiche 11 (modifié)
    return 0;
}
```

---

6. Pointeurs et Chaînes de Caractères

Les chaînes en C sont des tableaux de caractères terminés par le caractère nul (`\0`). Les pointeurs sont très utilisés pour manipuler les chaînes.

Exemple : Parcourir une chaîne avec un pointeur

```c
char *str = "Bonjour";
while (*str != '\0') {
    printf("%c ", *str);
    str++; // Passe au caractère suivant
}
```

---

7. Gestion Dynamique de Mémoire

En C, les pointeurs sont indispensables pour allouer et libérer de la mémoire dynamiquement.

a) Allocation dynamique

- `malloc` : Alloue un bloc mémoire non initialisé.
- `calloc` : Alloue un bloc mémoire initialisé à 0.

```c
int *ptr = (int *)malloc(5 * sizeof(int)); // Alloue un tableau de 5 entiers
if (ptr == NULL) {
    printf("Erreur d'allocation.\n");
    return 1;
}

// Initialiser les valeurs
for (int i = 0; i < 5; i++) {
    ptr[i] = i * 10;
    printf("%d ", ptr[i]);
}

// Libérer la mémoire
free(ptr);
```

b) Erreurs courantes avec la mémoire dynamique

- **Fuite mémoire** : Oublier de libérer la mémoire avec `free`.
- **Double libération** : Appeler `free` deux fois sur le même pointeur.
- **Accès à un pointeur non initialisé** : Ne pas initialiser le pointeur avant de l'utiliser.

---

8. Les erreurs courantes à éviter

1. **Pointeur non initialisé** :
    - Toujours initialiser un pointeur avant de l'utiliser.

    ```c
    int *ptr = NULL; // Initialisation sécurisée
    ```

2. **Déréférencement d'un pointeur NULL** :
   - Vérifiez que le pointeur n'est pas NULL avant de le déréférencer.

```
if (ptr != NULL) {
    printf("%d", *ptr);
}
```

3. **Accès à une mémoire libérée** :
   - Après avoir libéré un pointeur avec `free`, ne pas y accéder.

```
free(ptr);
ptr = NULL; // Bonne pratique
```

4. **Mauvais calcul d'adresse** :
   - Les pointeurs sont sensibles aux erreurs de calcul (exemple : dépasser la taille d'un tableau).

---

9. Visualisation pour la Compréhension

Exemple illustré :

```
int a = 5;
int *ptr = &a;
int **pptr = &ptr;

printf("Valeur de a : %d\n", a);       // 5
printf("Adresse de a : %p\n", &a);     // Adresse de a
printf("Valeur de ptr : %p\n", ptr);   // Adresse de a
printf("Valeur de *ptr : %d\n", *ptr); // 5
printf("Valeur de **pptr : %d\n", **pptr); // 5
```

---

10. Avantages et Limites des Pointeurs

Avantages :

- Accès direct à la mémoire.
- Passage par référence (modifie les variables originales).
- Indispensable pour la gestion dynamique de mémoire.

Limites :

- **Complexité** : Les erreurs de manipulation peuvent entraîner des bugs difficiles à corriger.

- **Fragilité** : Un mauvais calcul peut provoquer un crash (segmentation fault).
- **Sécurité** : Les pointeurs permettent d'accéder à n'importe quelle adresse mémoire, ce qui peut être dangereux.

Conclusion

Les pointeurs sont une des fonctionnalités les plus puissantes du langage C. En comprenant leur fonctionnement, leurs usages et leurs limites, vous pouvez :

- Manipuler efficacement des structures complexes.
- Optimiser la mémoire utilisée.
- Créer des programmes performants et dynamiques.

Pratiquez régulièrement pour éviter les erreurs courantes !